Practical No. 06

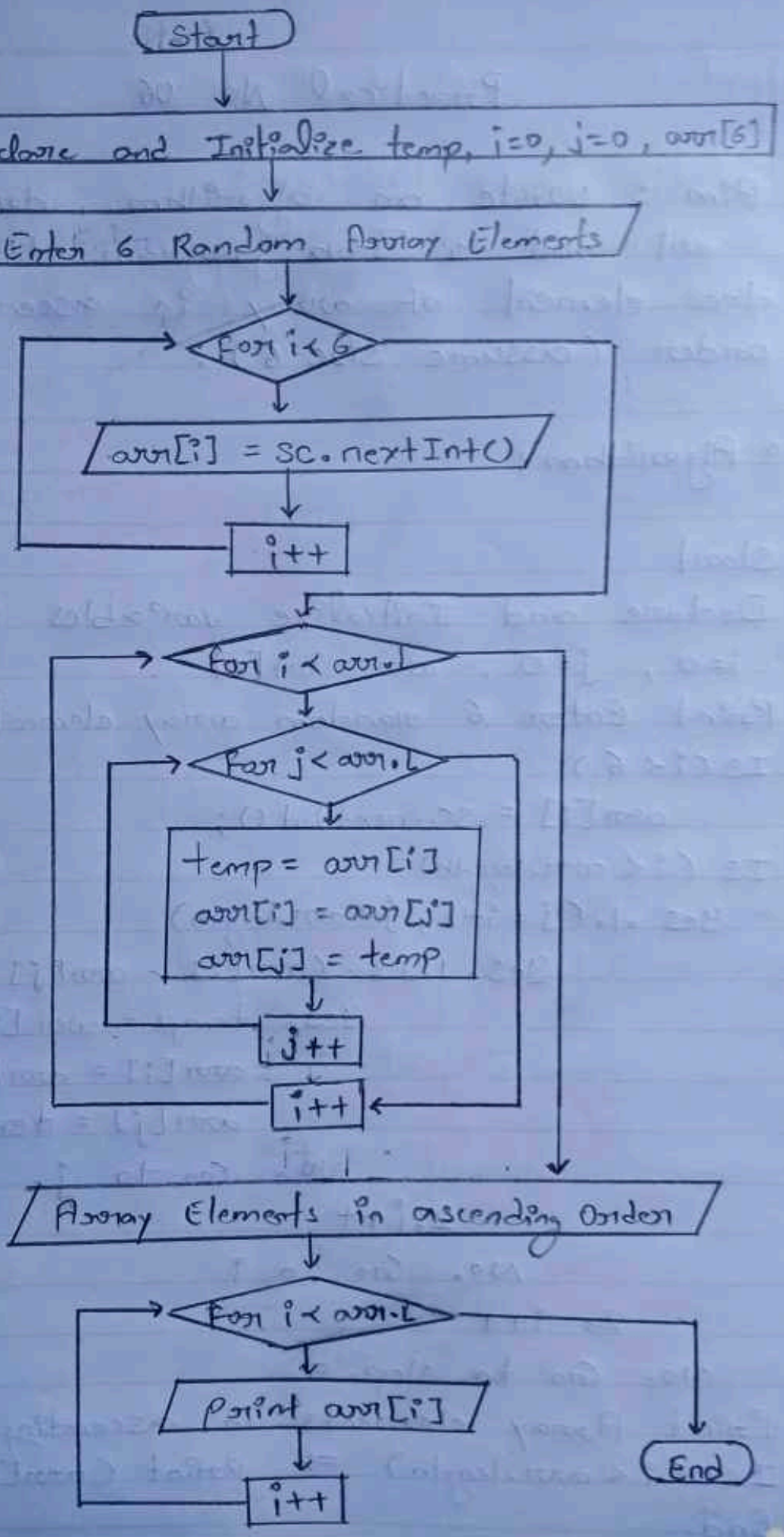
E.G Aim : Write an algorithm, draw Flowchart and a Java program to arrange the element of array in ascending order (assume size 6).

- Algorithm :

1. Start
2. Declare and Initialize variables temp, i=0, j=0, arr = int[6]
3. Print Enter 6 random array elements.
   Is (i < 6)
   arr[i] = sc.nextInt();
4. Is (i < arr.length)
   yes 1. (j = i+1, j < arr.length)
   yes. 1. Is (arr[i] > arr[j]
   yes. temp = arr[i]
   arr[i] = arr[j]
   arr[j] = temp
   No. Go to j.
   2. i++
   No. Go to i
   2. i++
   No. Go to step. 5.
5. Print Array elements in ascending order.
   Is (i < arr.length) ⇒ print (arr[i])
6. End.

nsg

Flowchart ;



nsg

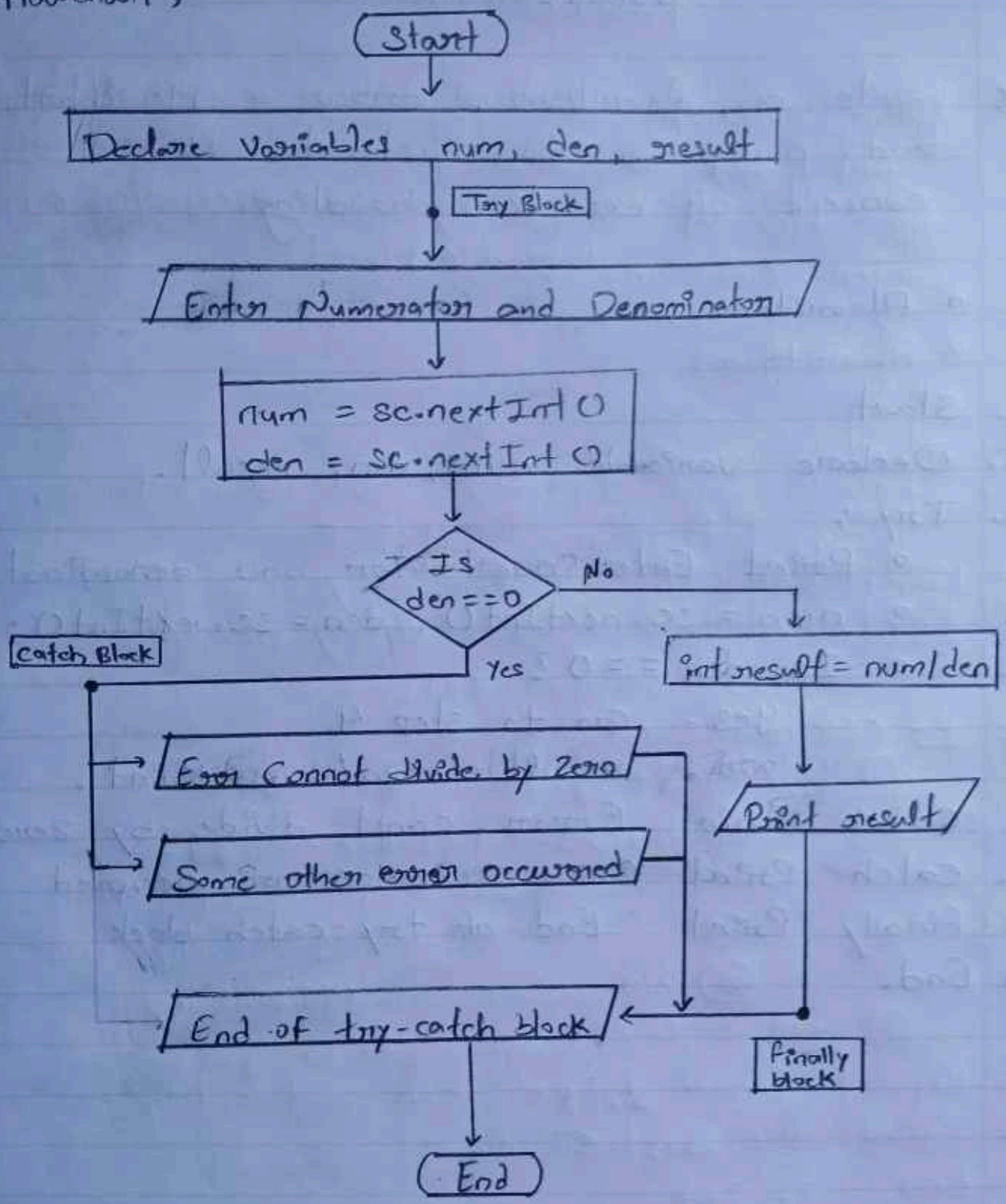
Practical No. 07.

E.7 Write an algorithm, draw a flowchart and program to illustrate the concept of exception handling.

- Algorithm ;

1. Start
2. Declare variable num, den, result.
3. Try ;
   - Print Enter numerator and denominator.
   - num = sc.nextInt(), den = sc.nextInt();
   - Is den == 0 ?
     - Yes. Go to step 4.
     - No. result = num/den, Print.
4. Catch Print Error, can't divide by zero.
5. Catch Print Some, other error occured
6. Finally Print " End of try-catch block.
7. End.

Flowchart ;



nsg

Practical No. 08

E. 8 Aim : Write a Java program to perform the concept of interface.

Program ;

```
Import java.util.Scanner;

public class interface_nitin {
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);

        System.out.println ("Enter radius
                of the circle :");
        doble radius = sc.nextDouble();
        Shape s1 = new Circle (radius);
        s1.getArea ();
        sc.close ();
    }
}

interface Shape {
    double PI = 3.14159;
    void getArea();
}

class Circle implements Shape {
    double radius;
```

```
Circle (double r) {
        radius = r;
    }
    public void getArea() {
        double area = PI * radius * radius
        System.out.println (" Area of
            circle = " + area);
    }
}
```

- Output :

```
Enter radius of the circle : 4
Area of circle = 50.26544
```

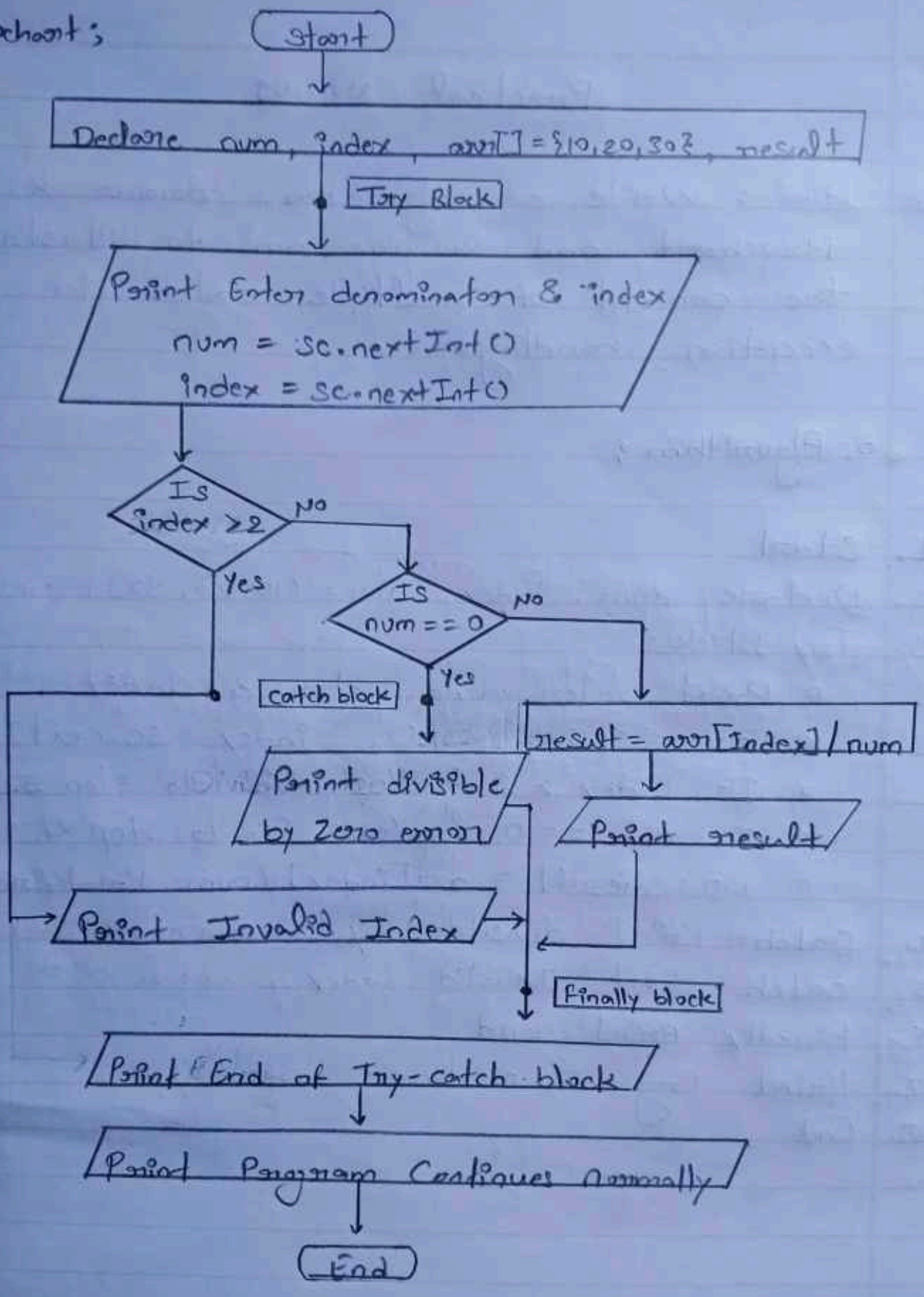
Practical No. 09.

E.9. Aim : Write an algorithm, draw a flowchart and a program to illustrate the concept of multiple catch in exception handling.

- Algorithm ;

1. Start
2. Declare num, index, arr = (10, 20, 30), result.
3. Try Block :
   - Print Enter denominator & index
   - num = sc.nextInt(), index = sc.nextInt()
   - Is index > 2 ? Yes : Go to step 5.
   - Is num == 0 ? Yes : Go to step 4.
   - NO : result = arr[index] / num, Print result.
4. Catch Print divisible by zero error
5. catch Print Invalid index.
6. Finally print End
7. Print Program continues normally
8. End.

Flowchart;



nsg

Practical No. 10

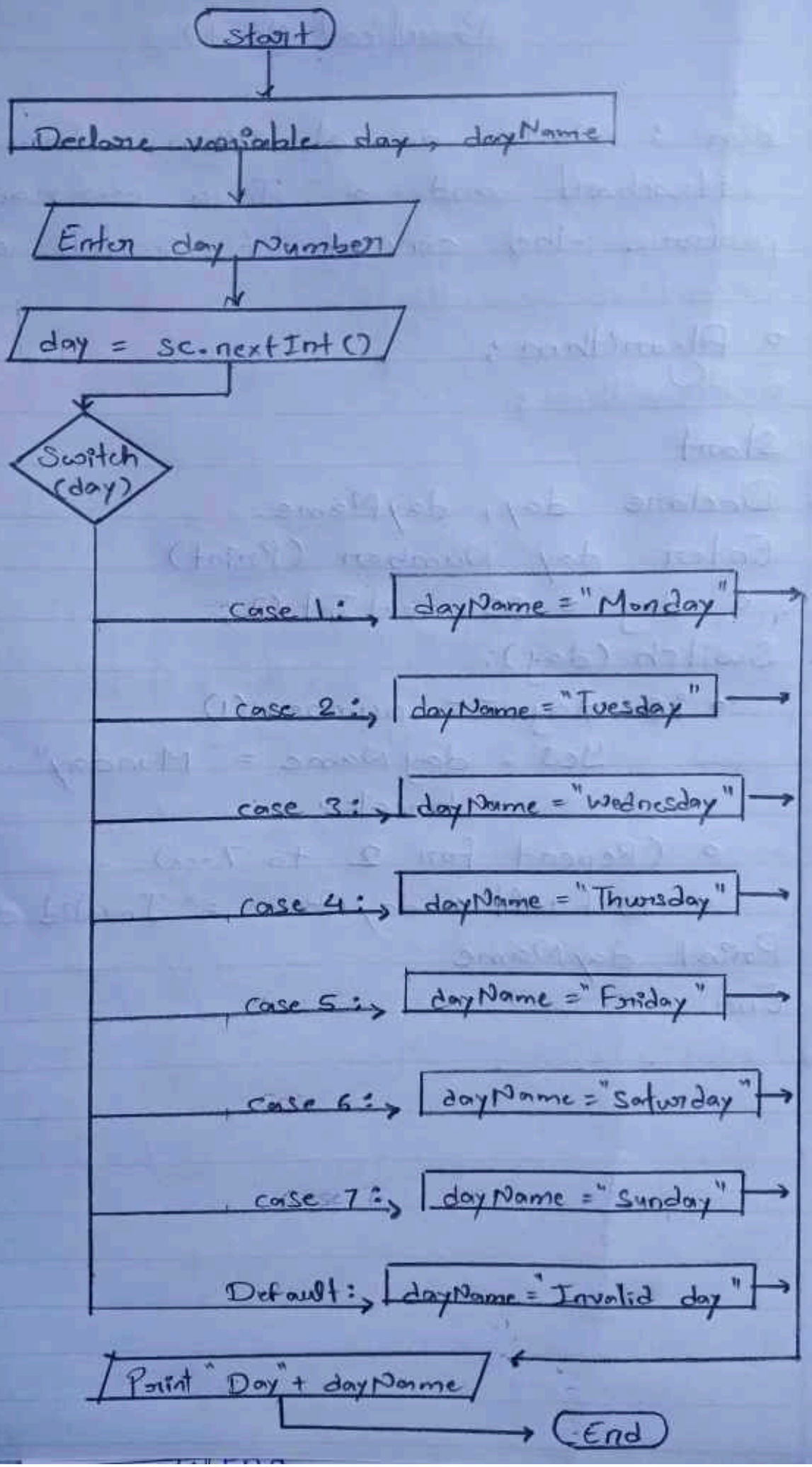
E.10 Aim : Write an algorithm, draw a flowchart and a Java program to perform the concept of switch case.

- Algorithm ;

1. Start
2. Declare day, dayName.
3. Enter day Number (Print)
   - day = sc.nextInt ();
4. Switch (day);
   - is day == number (1)
     Yes. dayName = "Monday".
     break;
   - (Repeat for 2 to 7...)
   - Default : dayName = "Invalid day".
5. Print dayName
6. End.

Kalsoni EDUCATION

Flowchart



nsg

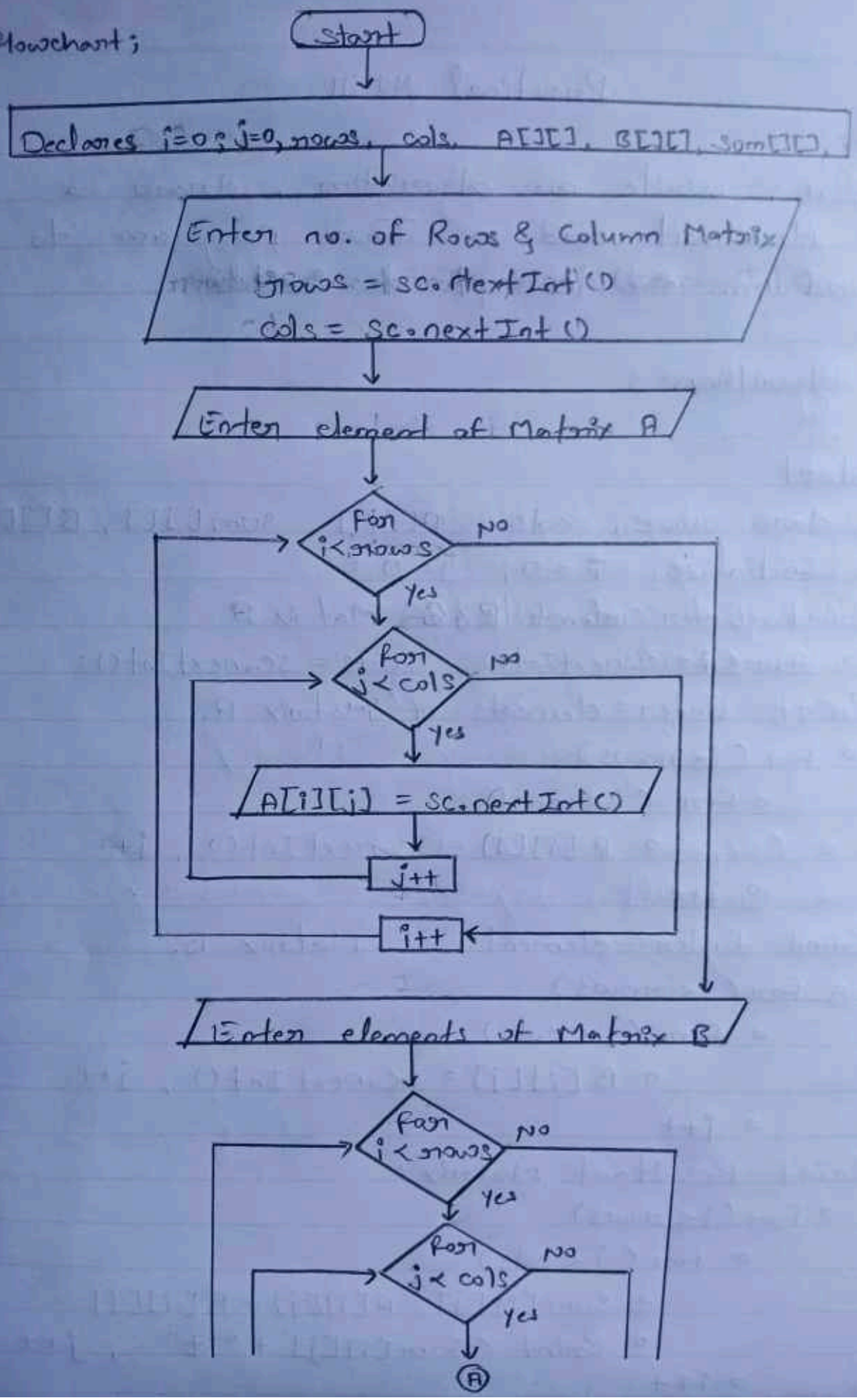
Practical No. 11

E.11 Aim : Write an algorithm, draw a flowchart and a Java program to perform addition on two matrix.

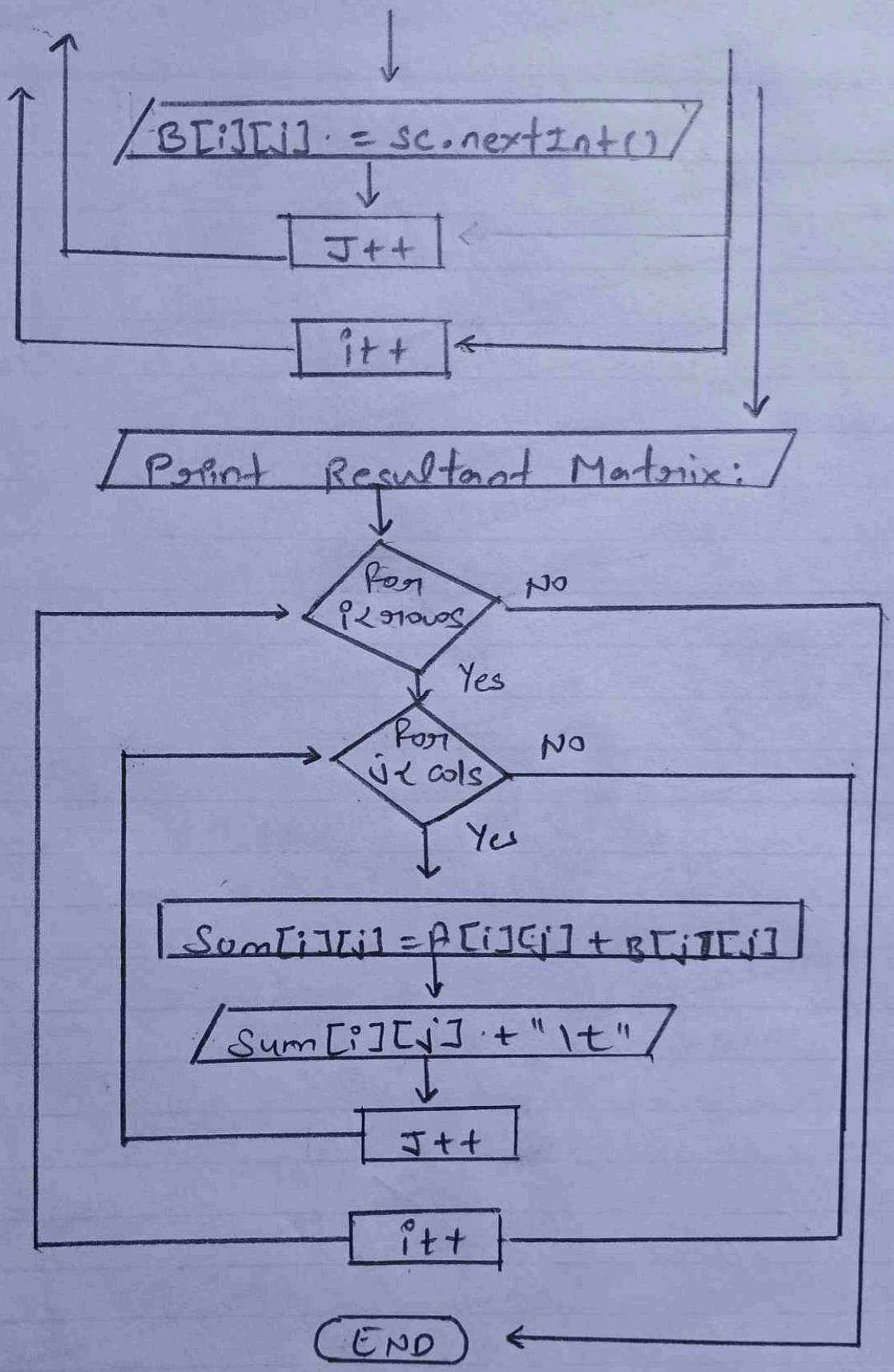
- Algorithm;

1. Start
2. Declare rows, cols, A[][], sum[][], B[][]
   Initialize, i = 0, j = 0.
3. Print Enter no. of R & C Matrix.
   - rows = sc.nextInt(), cols = sc.nextInt();
4. Print Enter elements of Matrix A.
   - for (i < rows)
     - for (j < cols)
       - A[i][j] = sc.nextInt(), j++
     - i++
5. Print Enter elements of Matrix B.
   - for (i < rows)
     - for (j < cols)
       - B[i][j] = sc.nextInt(), j++
     - i++
6. Print Resultant Matrix:
   - for (i < rows)
     - for (j < cols)
       - sum[i][j] = A[i][j] + B[i][j]
       - Print (sum[i][j] + "\t", j++
     - i++
7. End

Flowchart;



nsg


B[i][j] = sc.nextInt()
J++
i++
Print Resultant Matrix:
For i< rows
NO
Yes
For j< cols
NO
Yes
Sum[i][j] = A[i][j] + B[i][j]
Sum[i][j] + "\t"
J++
i++
END